BÁO CÁO THỰC HÀNH TURTLEBOT 4

Định vị trên bản đồ có sẵn, điều hướng tự động bằng Nav2 và nhận diện vật thể bằng YOLO

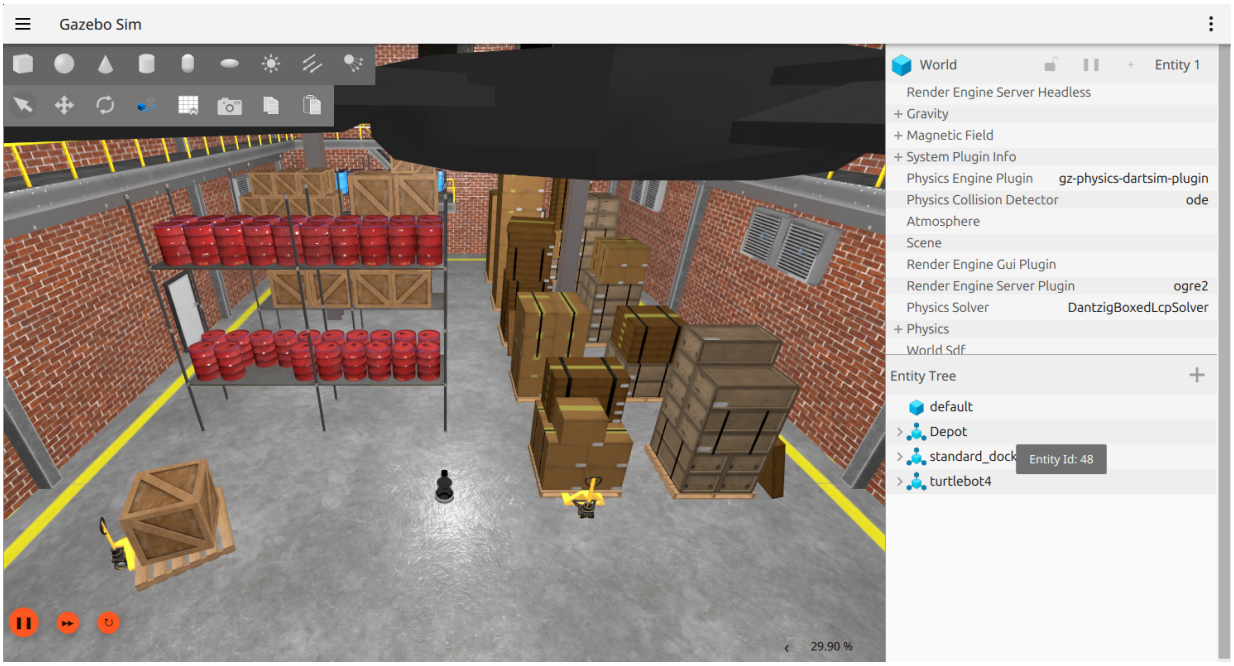
Ngày thực nghiệm: **20/10/2026** · Múi giờ Việt Nam (UTC+7).
Môi trường: Ubuntu 24.04, ROS 2 Jazzy, Gazebo Harmonic, TurtleBot 4 Standard, world Depot.

Họ và tên: **Nguyễn Minh Nghĩa**
Mã sinh viên: **23020755**
Lớp / Nhóm: **K68 E RE**

Kết quả thực nghiệm

Chuỗi navigation: **ĐẠT — 3/3 goal thành công đúng thứ tự**. Thời gian từ gửi Goal 1 đến kết quả Goal 3: **139.81 giây thực**.

AMCL: sai lệch vị trí ban đầu so với ground truth Gazebo **6.81 mm**. YOLO: 15 lần ghi log có nhãn chair; độ tin cậy cao nhất **0.86**.



Hình 1. Ảnh chụp trực tiếp Gazebo Depot và TurtleBot 4 trong phiên chạy ngày 20/10/2026, trước khi thêm ghế thử.

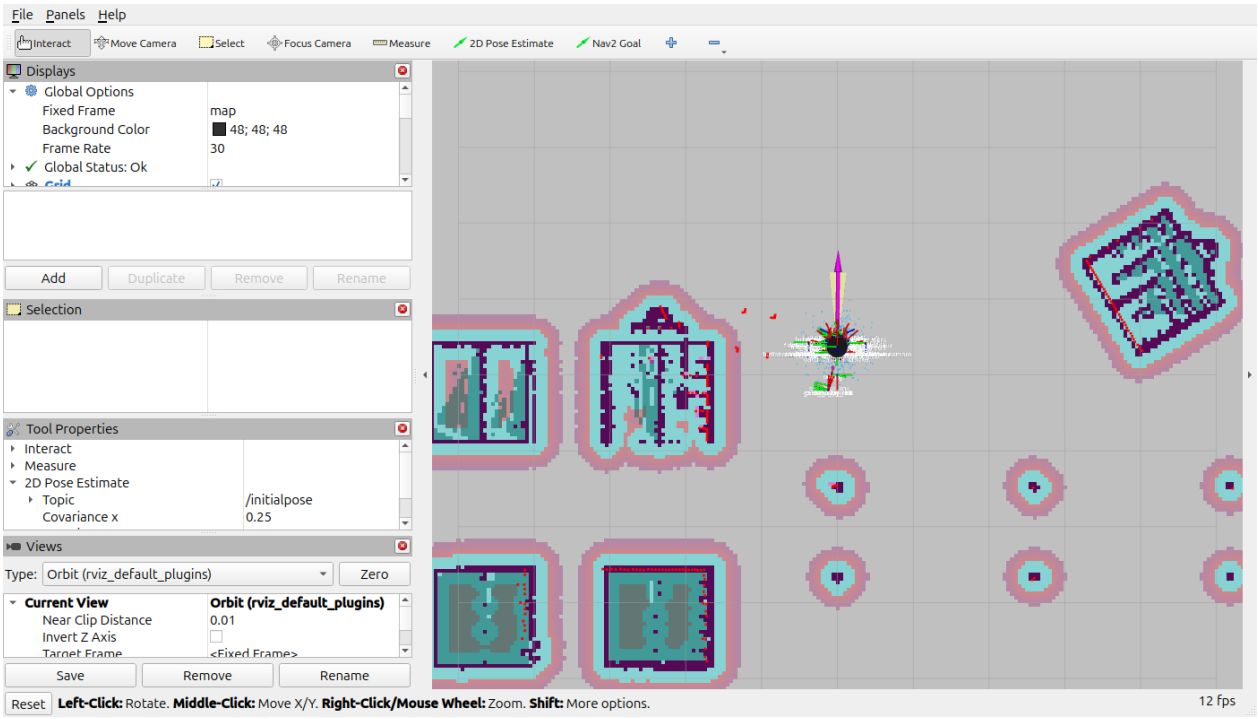
Toàn bộ ảnh và số liệu trong báo cáo được thu từ phiên chạy mới. Ground truth chỉ dùng để đặt và kiểm tra initial pose trong mô phỏng; node navigation vận hành bằng AMCL và Nav2.

1. Khởi chạy mô phỏng và định vị

Nạp lại map.yaml và map.pgm của workspace, không chạy SLAM đồng thời. Khởi động Gazebo, robot và bridge; tắt E-stop trong mô phỏng, gọi action /undock và đợi kết quả SUCCEEDED. Sau đó đặt initial pose theo vị trí/hướng thật của robot trong Gazebo bằng script phục vụ kiểm thử.

RViz dùng Fixed Frame = map. AMCL cung cấp biến đổi map → odom, kết hợp odom → base_link để Nav2 định vị robot. Kiểm tra nhận dữ liệu map, scan, odom, tf, tf_static, amcl_pose và cmd_vel; cả sáu lifecycle node đều active.

Thông số	Giá trị
Bản đồ	604 × 304 ô; độ phân giải 0,05 m/ô
AMCL ban đầu (x, y)	(-0.35748, -0.00268) m
Ground truth (x, y)	(-0.35122, -0.00000) m
Sai lệch vị trí ban đầu	0.006805 m
Lifecycle	amcl, map_server, planner_server, controller_server, bt_navigator, collision_monitor: active



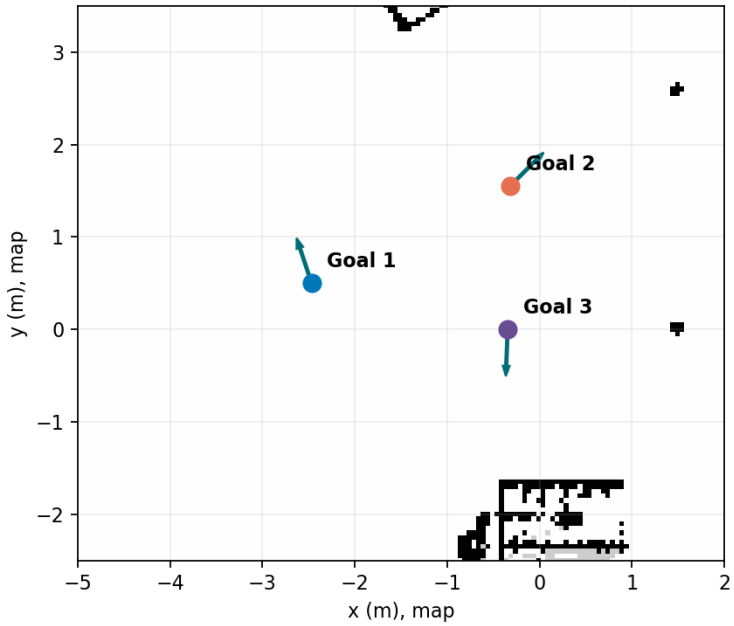
Hình 2. RViz sau khởi tạo AMCL: Fixed Frame map, trạng thái OK, hiển thị bản đồ, costmap, robot và dữ liệu LiDAR.

Lưu ý: sai lệch trên là phép kiểm tra tại thời điểm ban đầu, không phải kết luận về độ chính xác ở mọi vị trí. Ground truth không thay thế AMCL trong điều khiển.

2. Tọa độ mục tiêu và chương trình điều hướng

Sử dụng ba tọa độ đã được ghi nhận trong navigation_goals.yaml. Hai điểm đầu thuộc vùng trống trên map Depot; điểm thứ ba quay về khu vực xuất phát sau undock. Tọa độ thuộc frame map, đơn vị mét và radian. Cách chọn trước đó dựa trên ô trống, khoảng hở và đường planner; phiên này giữ nguyên tọa độ để kiểm tra lặp lại.

Mục tiêu	x (m)	y (m)	yaw (rad)
Goal 1	-2.468000	0.503000	1.892550
Goal 2	-0.318000	1.553000	0.785399
Goal 3	-0.345763	0.006710	-1.609538



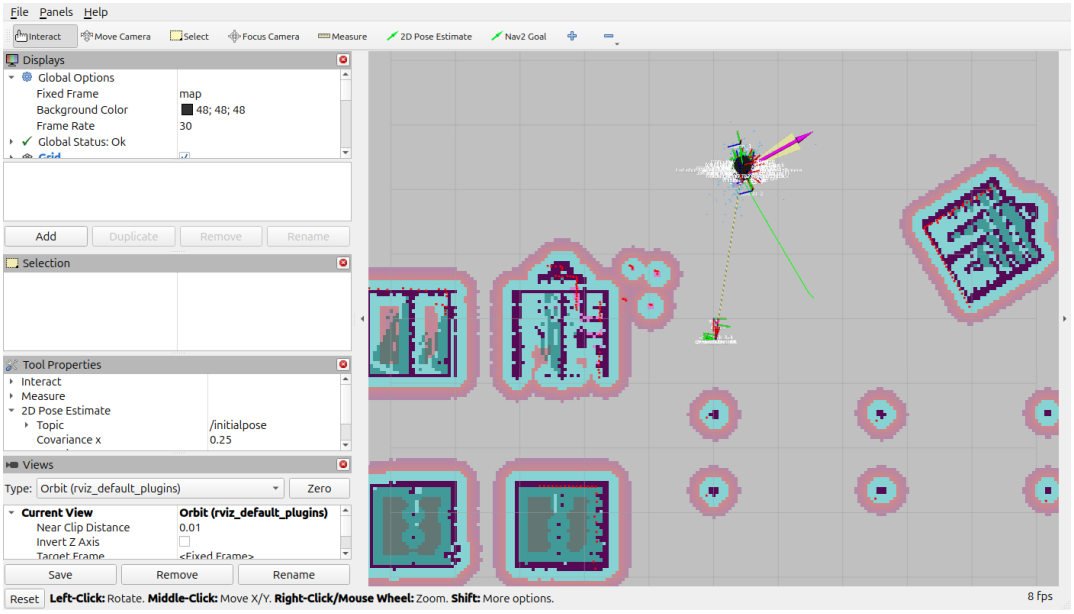
Hình 3. Vị trí và hướng của ba goal trên bản đồ lưu. Đây là hình minh họa từ PGM và YAML, không phải ảnh chụp RViz hay đường robot đã đi.

Node navigation_node.py kế thừa BasicNavigator. Chương trình đợi Nav2 active, AMCL có pose, đồng hồ mô phỏng hoạt động và robot đã rời dock; sau đó tạo PoseStamped trong map và gọi NavigateToPose cho từng goal. Quaternion hướng được tính bằng z = sin(yaw/2), w = cos(yaw/2).

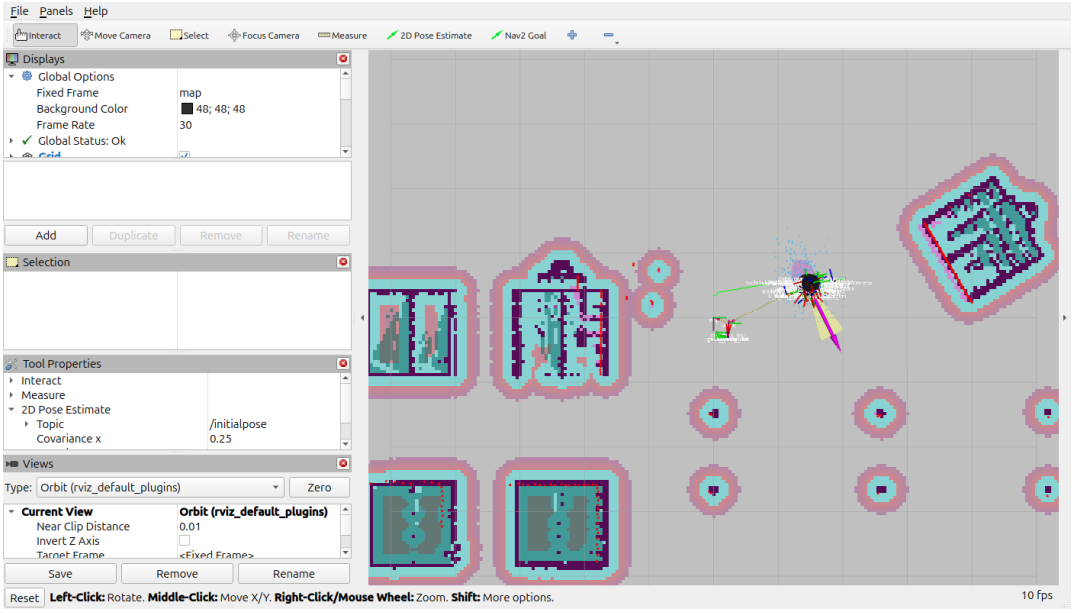
Chỉ gửi goal tiếp theo khi TaskResult.SUCCEEDED. Nếu bị từ chối, thất bại hoặc hết thời gian, chương trình dừng chuỗi và yêu cầu hủy action. Trong chuỗi thử nghiệm không sử dụng teleop và không đặt goal bằng chuột trên RViz.

3. Kết quả ba goal tự động

Goal	Kết quả action	Thời gian chặng (giây thực)
Goal 1	SUCCEEDED	55.96
Goal 2	SUCCEEDED	49.20
Goal 3	SUCCEEDED	34.66

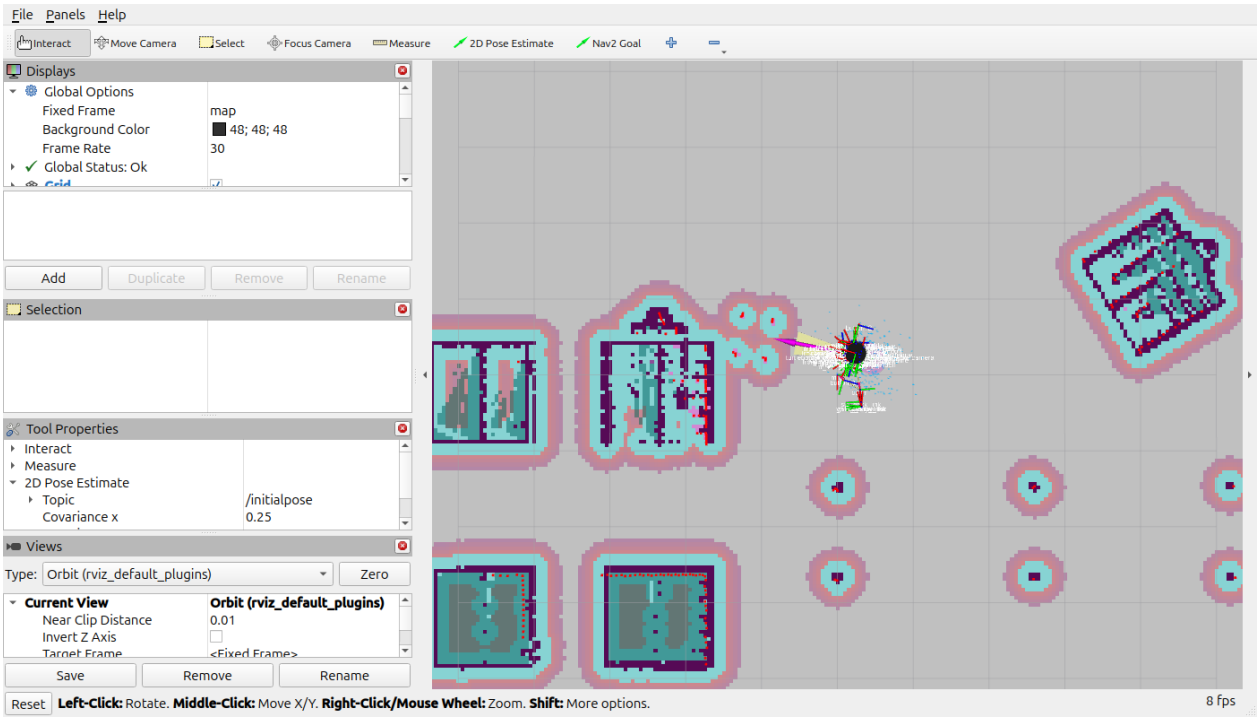


Hình 4. RViz được chụp sau khi log báo Goal 1 succeeded; node tự động chuyển sang goal kế tiếp nên ảnh có thể đã hiển thị đường đi chặng tiếp theo.



Hình 5. RViz được chụp sau khi log báo Goal 2 succeeded; node tự động chuyển sang goal kế tiếp nên ảnh có thể đã hiển thị đường đi chặng tiếp theo.

4. Hoàn tất hành trình và kiểm tra va chạm



Hình 6. RViz sau khi Goal 3 thành công, kết thúc chuỗi điều hướng tự động.

Trong cửa sổ giám sát 185.63 giây thực, nhận 2226 message hazard_detection và 0 message bumper_contact. Số lần quan sát hazard BUMP: **0**; số message contact không rỗng: **0**.

Luồng hazard_detection có dữ liệu thực và không ghi nhận BUMP trong cửa sổ thử nghiệm. Kênh bumper_contact không nhận message; số 0 ở kênh này không được coi là xác nhận luồng contact hoạt động. Giới hạn: hazard bumper không chứng minh mọi tiếp xúc hình học ở mọi bộ phận robot; không đánh đồng action thành công với bằng chứng tuyệt đối không va chạm.

```
[NAV] Navigating to Goal 1
[NAV] Goal 1 succeeded
[NAV] Navigating to Goal 2
[NAV] Goal 2 succeeded
[NAV] Navigating to Goal 3
[NAV] Goal 3 succeeded
[NAV] All navigation goals completed successfully
```

Minh chứng gốc: combined_launch.log, collision_summary.json, interfaces.json và các ảnh goal_1_rviz.png đến goal_3_rviz.png trong thư mục evidence của phiên này.

5. Nhận diện vật thể từ camera bằng YOLO

Node yolo_detection_node.py subscribe /oakd/rgb/preview/image_raw bằng sensor-data QoS. CvBridge chuyển sensor_msgs/Image sang ảnh OpenCV bgr8. YOLOv8n chạy trên CPU (ngưỡng confidence 0,4; imgsz 320; mục tiêu 2 Hz); inference ở worker riêng để không chặn callback ảnh hay node navigation.

Kết quả được vẽ bounding box, tên class, confidence bằng result.plot và publish lên /yolo/detections/image. Terminal ghi class/confidence theo chu kỳ. RGB và annotated đều 320 × 240; ảnh đầu vào rgb8, ảnh đầu ra bgr8.

Thêm model Chair static từ cache Fuel vào Gazebo trước khi chạy chuỗi goal, tại x = −0,52 m, y = −1,03 m, yaw = 1,39 rad. Đây là vật thể 3D trong môi trường mô phỏng; không dùng ảnh ghi sẵn làm đầu vào. Fixture được kế thừa từ lần thử trước, chỉ tồn tại trong phiên Gazebo, chưa cố định vào world.



Hình 7. Ảnh camera trực tiếp được node YOLO chú thích trong phiên thử mới. Tên class và confidence nằm trên bounding box.

Trong log của phiên tích hợp có **15** lần ghi nhận nhãn chair, confidence cao nhất **0.86**. YOLO chạy đồng thời với navigation; một số góc nhìn có 0 detections là bình thường.

Giới hạn nhận diện: mô hình pretrained COCO có khác biệt miền ảnh với Gazebo; không bảo đảm nhận đúng mọi vật thể. Log còn có nhãn umbrella; các nhãn này không được coi là xác nhận vật thể thật, có thể là false positive.

6. Tổng hợp và cách tái hiện

Hạng mục	Kết quả / minh chứng
Build workspace	PASS — build.log
Unit tests	2/2 PASS — unit_tests.log
Map + AMCL + TF	PASS — localization.json, interfaces.json
Navigation tuần tự	3/3 SUCCEEDED — combined_launch.log
Camera + cv_bridge + YOLO	Ảnh đầu vào/đầu ra có message, có nhận diện chair
Va chạm bumper	BUMP=0; contact không rỗng=0
Bảo toàn map	SHA-256 được lưu trong summary.json; đối chiếu bản test trước

Ba terminal đều source /opt/ros/jazzy/setup.bash và install/setup.bash trong workspace. Thực hiện undock và đặt initial pose trước khi bắt đầu nhiệm vụ. Thêm lại fixture ghế nếu muốn tái hiện nhận diện dương tính.

```
# Terminal 1
ros2 launch tb4_simulation_lab gazebo_rviz.launch.py joy:=false
# Terminal 2
ros2 launch tb4_simulation_lab localization_navigation.launch.py
# Chu□n bị: /undock; đặt initial pose trong RViz
# Terminal 3
source .venv-yolo/bin/activate
ros2 launch tb4_simulation_lab autonomous_yolo.launch.py \
  goals_file:=/home/nghia/turtlebot4/src/tb4_simulation_lab/config/navigation_goals.yaml
    \
  yolo_model:=/home/nghia/turtlebot4/models/yolov8n.pt show_window:=false
```

Lưu ý khi dừng: sau khi navigation hoàn tất, thao tác Ctrl+C kết thúc detector gây KeyboardInterrupt trong bước destroy_publisher (exit −2). Inference và navigation đã hoàn thành trước đó; đây là hạn chế cleanup khi tắt launch, chưa được sửa trong phiên đánh giá này. Log gốc được giữ nguyên.

Mã nguồn: src/tb4_simulation_lab/scripts/navigation_node.py và yolo_detection_node.py. Cấu hình: config/navigation_goals.yaml, config/nav2_params.yaml. Launch kết hợp: launch/autonomous_yolo.launch.py.

Minh chứng phiên mới: docs/evidence/integration_2026-10-20/. Script tạo PDF và dữ liệu JSON nằm cùng thư mục để kiểm tra lại số liệu. Ảnh được chụp từ cửa sổ thật hoặc nhận từ topic camera; hình bản đồ mục tiêu được ghi rõ là hình minh họa.

Kết luận: phiên chạy kiểm tra được việc sử dụng bản đồ có sẵn để định vị, tự động hoàn thành ba goal bằng ROS 2 Node và nhận diện ghế từ camera trong lúc hệ thống vận hành. Nguồn gốc bản đồ của bài tuần trước cần được người thực hiện xác nhận trước khi nộp.